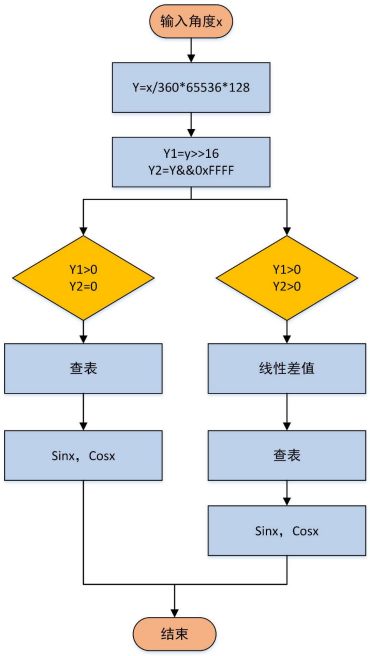
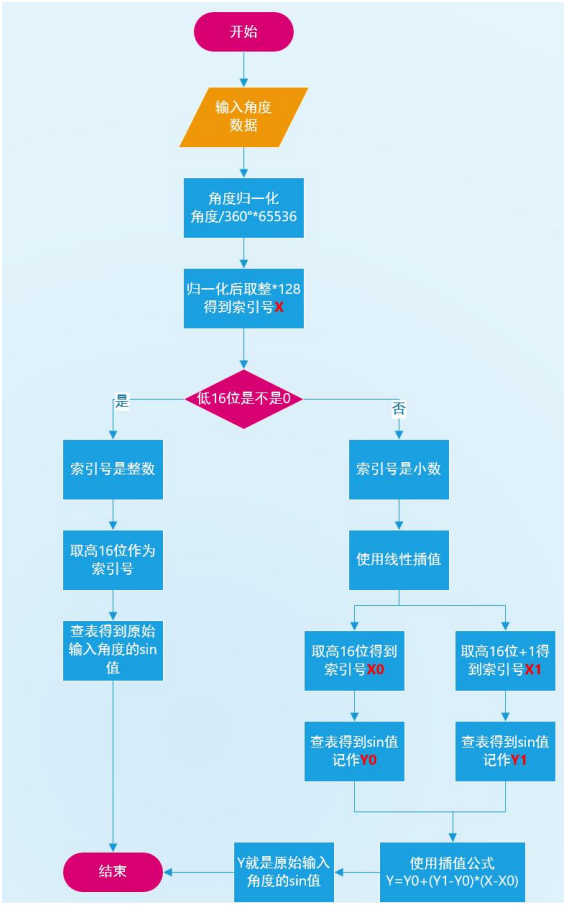
40-SVPWM 11

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

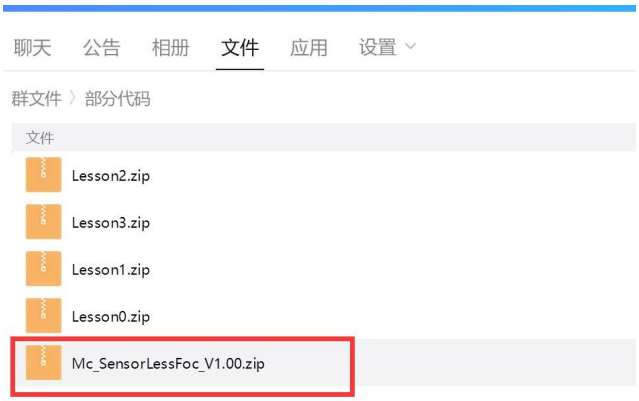


上节课，安排作业把查表法的实现流程图给画出来。前面我们看到板栗学员上传的图，我们谢谢板栗！大家觉得流程有没有问题？我看思路基本都是正确的，我再贴一个往期学员的图给大家看一下。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



这样呢，大家根据所绘制的流程图去编写相应的程序，思路应该就很清晰了。那我们今天呢，就先来分析一下相应的查表法的函数。



今晚引用的代码在红框文件中有相应的代码。

```
/*=============================================================================*/
static void CalSinCos(unsigned int EAngle)
{
    unsigned int Temp;
    unsigned int *p1;
    unsigned int *p2;
    signed int Y0;
    signed int Y1;
    unsigned char TempIndex;

    tCore.Index.U32 = __builtin_muluu(EAngle,128);
    if(tCore.Index.Words.Low == 0)
    {
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

大家看啊，这个函数呢，它含有一个形参。这个形参呢，就是我们所讲的 θ，这里呢，角度跟归一化的这个过程，不用编程来实现，而靠我们编程者自己定义好。因为我们心里是清楚的，0~360° 的角度，归一化之后转换为(0.16)数据格式的话，就对应 0~65535。所以，我们在编程的时候，已经默认角度数据已经是处理好了。这样的话，我们就可以直接使用了因此啊，这个函数的形参也就是输入的角度，对应的范围就是 0~65535。

上面的流程图，把归一化及数据格式转换的步骤也写出来，表示思路都是清楚的了。我们这里统一一下，转换不需要我们在程序中去转换，因为定义数据格式的时候已经隐含转换了。然后，我们接着来分析啊。

```
/*=====================================================================
static void CalSinCos(unsigned int EAngle)
{
    unsigned int Temp;
    unsigned int *p1;
    unsigned int *p2;
    signed int Y0;
    signed int Y1;
    unsigned char TempIndex;

    tCore.Index.U32 = __builtin_muluu(EAngle,128);
    if(tCore.Index.Words.Low == 0)
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(SinTable);
        p1 = (unsigned int *)__builtin_psvoffset(SinTable);
        p2 = p1 + tCore.Index.Words.High;
        tCore.qSin = *p2;

        TempIndex = tCore.Index.Words.High + 32;
        if(TempIndex > 127)
            TempIndex = TempIndex - 128;
        p2 = p1 + TempIndex;
        tCore.qCos = *p2;
        DSRPAG = Temp;
    }
```

那么，函数一进来，我们就可以求解角度跟表大小(128)的乘积。这个乘积呢，是作为索引号使用的。

```
const signed int SinTable[128] = {
0,      1608,   3212,   4808,   6393,   7962,   9512,   11039,
12539,  14010,  15446,  16846,  18204,  19519,  20787,  22005,
23170,  24279,  25329,  26319,  27245,  28105,  28898,  29621,
30273,  30852,  31356,  31785,  32137,  32412,  32609,  32728,
32767,  32728,  32609,  32412,  32137,  31785,  31356,  30852,
30273,  29621,  28898,  28105,  27245,  26319,  25329,  24279,
23170,  22005,  20787,  19519,  18204,  16846,  15446,  14010,
12539,  11039,  9512,   7962,   6393,   4808,   3212,   1608,
0,      -1608,  -3212,  -4808,  -6393,  -7962,  -9512,  -11039,
-12539, -14010, -15446, -16846, -18204, -19519, -20787, -22005,
-23170, -24279, -25329, -26319, -27245, -28105, -28898, -29621,
-30273, -30852, -31356, -31785, -32137, -32412, -32609, -32728,
-32767, -32728, -32609, -32412, -32137, -31785, -31356, -30852,
-30273, -29621, -28898, -28105, -27245, -26319, -25329, -24279,
-23170, -22005, -20787, -19519, -18204, -16846, -15446, -14010,
-12539, -11039, -9512,  -7962,  -6393,  -4808,  -3212,  -1608};
```

这个表呢，就是我们上次课讲过的，128 个点转换为 Q15 格式的。直接从 excel 把数据复制过来，然后定义一个数组。然后，这个乘积呢，是一个 32 位的数据。这个 32 位的数据呢，高 16 位是索引号的整数部分，低 16 位，是索引号的小数部分。我们是否要进行线性插值计算，则就是根据这个索引号的低 16 位是否为 0 来判断的。

问：这个表为什么不用短整形，空间省一半？

答：当前编译器使用的是 16 位单片机，unsigned int 与 unsigned short int 都表示 16 位，　这一点与 32 位单片机不同。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：负值在程序里面怎么处理？
答：不需要特殊处理，C 语言定义有符号数可以直接使用。

```
/*=====================================================================
static void CalSinCos(unsigned int EAngle)
{
    unsigned int Temp;
    unsigned int *p1;
    unsigned int *p2;
    signed int Y0;
    signed int Y1;
    unsigned char TempIndex;

    tCore.Index.U32 = __builtin_muluu(EAngle,128);
    if(tCore.Index.Words.Low == 0)
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(SinTable);
        p1 = (unsigned int *)__builtin_psvoffset(SinTable);
        p2 = p1 + tCore.Index.Words.High;
        tCore.qSin = *p2;

        TempIndex = tCore.Index.Words.High + 32;
        if(TempIndex > 127)
            TempIndex = TempIndex - 128;
        p2 = p1 + TempIndex;
        tCore.qCos = *p2;
        DSRPAG = Temp;
    }
```

大家看啊，如果说这个低 16 位是 0，则我们直接通过查表就可以获取到正弦值，将索引号偏移 32 就可以查表获得余弦值。也就是说，进入到这个 if 分支后，主要执行查表操作。

那查表怎么查呢?

```
/*=====================================================================
static void CalSinCos(unsigned int EAngle)
{
    unsigned int Temp;
    unsigned int *p1;
    unsigned int *p2;
    signed int Y0;
    signed int Y1;
    unsigned char TempIndex;

    tCore.Index.U32 = __builtin_muluu(EAngle,128);
    if(tCore.Index.Words.Low == 0)
    {
        Temp = DSRPAG;
        DSRPAG = __builtin_psvpage(SinTable);
        p1 = (unsigned int *)__builtin_psvoffset(SinTable);
        p2 = p1 + tCore.Index.Words.High;
        tCore.qSin = *p2;

        TempIndex = tCore.Index.Words.High + 32;
        if(TempIndex > 127)
            TempIndex = TempIndex - 128;
        p2 = p1 + TempIndex;
        tCore.qCos = *p2;
        DSRPAG = Temp;
    }
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

查表的话，主要是通过红框中这几句话实现的。这里呢，因为我们存储的正弦数据是放在 FLASH 中的。然后呢，我们单片机取出 FLASH 中的数据，需要用到一些特殊的查表操作这里呢，又涉及到两个内建函数的使用。首先呢，DSRPAG 这个寄存器，是读页寄存器。我们单片机的 FLASH 呢，被分为了很多页。我们要查表的话，需要知道表是存储到哪一页的所以，就需要通过一个内建函数，把正弦表存储的页给获取到。

```
tCore.Index.U32 = __builtin_muluu(EAngle,128);
if(tCore.Index.Words.Low == 0)
{                                          获取正弦表存储的页
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;

    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    tCore.qCos = *p2;
    DSRPAG = Temp;
}
```

也就是这句话。读取出来的页数呢，就会存储到 DSRPAG 寄存器中。用于辅助我们访问正弦表数据。接着呢，虽然知道了正弦表存储的页，但是正弦表不一定就是从这个页的起始地址开始存放的。所以啊，我们需要把正弦表在这个页的页内起始地址给获取到。也就是，首先知道正弦表存储在哪一页，接着要知道表的第一个数据是从页内的哪个地址开始存储的。

```
tCore.Index.U32 = __builtin_muluu(EAngle,128);
if(tCore.Index.Words.Low == 0)
{
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;
```

所以，这两句话的意思就是，先获取正弦表存储的页，再获取正弦表在这个页的起始存放地址，这个地址就存储在 p1 指针中。也就是说，通过这两个内建函数，我们就获取到了正弦表的第一个数据的首地址。这个步骤呢，也仅是我们使用的这个单片机需要这样操作。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
tCore.Index.U32 = __builtin_muluu(EAngle,128);
if(tCore.Index.Words.Low == 0)
{
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;

    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    tCore.qCos = *p2;
    DSRPAG = Temp;
}
```

那么，正弦表的起始地址知道了，查表的索引号也知道了，就可以知道我们要查表的数据对应的地址了。所以，上述这句话，P2 指针中，就存储了要查表的目标正弦数据的地址。

问：只是读正弦表的存储页，不用通过页地址+表在当前页的偏移量就可以确定表的首地址吗？

答：不可以哦， 目标正弦数据的地址=表首地址（第一个数据地址）+表中的目标数据的偏移量【页数存储到 DSRPAG 中是内部硬件需要】。

问：表就是放在页的第一个地址吗？

答：不是，表首地址 是当前页中的偏移量（程序中的 p1）。

问：也就是 p1 读到的偏移地址已经加了 DSRPAG 读到的页地址了，是这样吗？

答：不是“加”，是指后面访问的偏移量是 当前页 中的偏移量。

问：不能直接通过数组名称获取数组的地址吗？

答：可以，但速度慢(相当于访问 FLASH)，没有这种方法快（相当于访问 RAM）。

问：用张飞的单片机，是不是就可以直接读？

答：是的，可以的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
tCore.Index.U32 = __builtin_muluu(EAngle,128);
if(tCore.Index.Words.Low == 0)
{
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;

    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    tCore.qCos = *p2;
    DSRPAG = Temp;
}
else
```

然后呢，要获取这个地址中的数据，就直接把 *p2 赋值给对应变量。这样的话，也就获取到了查表正弦值。

```
tCore.Index.U32 = __builtin_muluu(EAngle,128);
if(tCore.Index.Words.Low == 0)
{
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;

    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    tCore.qCos = *p2;
    DSRPAG = Temp;
}
```

所以，这段查表代码，大家看下，可以理解吧？好，理解的话，我们继续来看余弦值的获取。实际上，我们从正弦波跟余弦波的波形来看，它们是存在 90° 的相位差的。既然存在 90° 的相位差，我们就知道，它们的正弦值相差多少。当 θ = 0° 的时候，sin θ = 0，cos θ = 1。所以，当 sin θ = 0 的时候，cos θ = 32767（Q15）。

```
/*=============================================================
const signed int SinTable[128] = {
0,      1608,   3212,   4808,   6393,   7962,   9512,   11039,
12539,  14010,  15446,  16846,  18204,  19519,  20787,  22005,
23170,  24279,  25329,  26319,  27245,  28105,  28898,  29621,
30273,  30852,  31356,  31785,  32137,  32412,  32609,  32728,
32767,  32728,  32609,  32412,  32137,  31785,  31356,  30852,
30273,  29621,  28898,  28105,  27245,  26319,  25329,  24279,
23170,  22005,  20787,  19519,  18204,  16846,  15446,  14010,
12539,  11039,  9512,   7962,   6393,   4808,   3212,   1608,
0,      -1608,  -3212,  -4808,  -6393,  -7962,  -9512,  -11039,
-12539, -14010, -15446, -16846, -18204, -19519, -20787, -22005,
-23170, -24279, -25329, -26319, -27245, -28105, -28898, -29621,
-30273, -30852, -31356, -31785, -32137, -32412, -32609, -32728,
-32767, -32728, -32609, -32412, -32137, -31785, -31356, -30852,
-30273, -29621, -28898, -28105, -27245, -26319, -25329, -24279,
-23170, -22005, -20787, -19519, -18204, -16846, -15446, -14010,
-12539, -11039, -9512,  -7962,  -6393,  -4808,  -3212,  -1608};
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

而从表中我们可以知道，同一个角度，对应的正弦值跟余弦值的索引之差是 32，也就是说，正弦值的索引是 0 的时候，余弦值的索引是 32。也可以这样看，0-128 对应 0-360 度， 360 度/4 = 90 度， 128/4 = 32， 而正弦和余弦相位相差 90 度。这样的话，我们就可以通过这个关系来查表获取输入角度对应的余弦值。

```
tCore.Index.U32 = __builtin_muluu(EAngle,128);
if(tCore.Index.Words.Low == 0)
{
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;

    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    tCore.qCos = *p2;
    DSRPAG = Temp;
}
else
```

所以我们来看代码，就是先在正弦值的索引号的基础上加 32，但是，加了 32 的话，我们还要保证索引号在 0~127 的范围内。所以啊，需要判断一下，是否索引号大于 127 了，如果大于的话，直接减 128 转换到 0~127 范围内来。这样的话，就获取到了余弦值的索引号。

```
if(tCore.Index.Words.Low == 0)
{
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;

    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    tCore.qCos = *p2;
    DSRPAG = Temp;
}
```

获取之后呢，还是先把地址给求取出来，然后再赋值。这样，余弦值就求查表出来了。

```
tCore.Index.U32 = __builtin_muluu(EAngle,128);
if(tCore.Index.Words.Low == 0)
{
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;

    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    tCore.qCos = *p2;
    DSRPAG = Temp;
}
```

这两句是成对出现的。先是保存现场，接着是恢复现场。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：为什么要这么做？

答：如果该函数是中断中使用的，假设主程序中也使用了 DSRPAG，该寄存器就需要保护和恢复现场。

问：*加地址还是变量？

答：星号是指针变量。

这个内建函数呢，大家可以查阅手册获取





__builtin_muluu

说明：

计算乘积 *p0* x *p1*。函数参数为无符号整型，函数结果为无符号长整型。可以使用命令行选项 -Wconversions 来检测意外的符号转换。该函数支持指令的所有寻址模式，包括操作数 *p1* 的立即数寻址模式。

原型：

```
unsigned long __builtin_muluu(const unsigned int p0, const unsigned int p1);
```

参数：

p0 被乘数
p1 乘数

返回值：

返回乘积 *p0* x *p1* 的无符号长整型值。

汇编操作符 / 机器指令：

```
mul.uu
```

然后，可以找到这个内建函数的使用说明。其实，就相当于是我们单片机提供的一个数学函数库。这个函数呢，是使用汇编写的。我们呢，就直接调用就可以。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：所有单片机都有吗？
答：不是，不同的单片机不一样。

__builtin_psvoffset

说明：	返回地址作为参数给出的对象的 psv 页偏移量。参数 *p* 必须是处于 EEdata、PSV 或可执行存储空间中的对象的地址；否则会产生错误消息，并且编译会失败。请参见**第 2.3.1 节“指定变量的属性”**中的 space 属性。
原型：	`unsigned int __builtin_psvoffset(const void *p);`
参数：	*p* 对象地址
返回值：	返回地址作为参数给出的对象的 psv 页偏移量。
汇编操作符 / 机器指令：	psvoffset
错误消息	如果该函数的使用不正确，则会产生以下错误消息： “Argument to __builtin_psvoffset() is not the address of an object in code, psv, or eedata section”（__builtin_psvoffset() 的参数不是处于代码、psv 或 eedata 段中的对象的地址）。 参数必须为显式对象地址。 例如，如果 *obj* 是一个处于可执行或只读段中的对象，则以下语法是有效的： `unsigned page = __builtin_psvoffset(&obj);`

p1 = (unsigned int *)__builtin_psvoffset(SinTable)这句话是得到正弦表起始地址在 FLASH 某个页中的偏移地址。

```
tCore.Index.U32 = __builtin_muluu(EAngle,128);
if(tCore.Index.Words.Low == 0)
{
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    tCore.qSin = *p2;

    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    tCore.qCos = *p2;
    DSRPAG = Temp;
}
else
{
    //Sin
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    Y0 = *p2;
    TempIndex = tCore.Index.Words.High + 1;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    Y1 = *p2;
    tCore.qSin = Y0 + (__builtin_mulus(tCore.Index.Words.Low,(Y1 - Y0)) >> 16);

    //Cos
```

然后，如果索引的低 16 位不是 0.　此时，也就说明索引号出现了小数，即是输入的角度，在正弦表中找不到对应的值。这样的话，我们就要进行线性插值计算。

```
//Sin
Temp = DSRPAG;
DSRPAG = __builtin_psvpage(SinTable);
p1 = (unsigned int *)__builtin_psvoffset(SinTable);
p2 = p1 + tCore.Index.Words.High;
Y0 = *p2;
TempIndex = tCore.Index.Words.High + 1;
if(TempIndex > 127)
    TempIndex = TempIndex - 128;
p2 = p1 + TempIndex;
Y1 = *p2;
tCore.qSin = Y0 + (__builtin_mulus(tCore.Index.Words.Low,(Y1 - Y0)) >> 16);
```

计算插值呢，还是需要把这个角度　前后在表中的两个正弦值给获取到。这个 Y0 呢，也就是输入角度　前面在表中的正弦值。查表的方法跟刚才讲的一模一样。

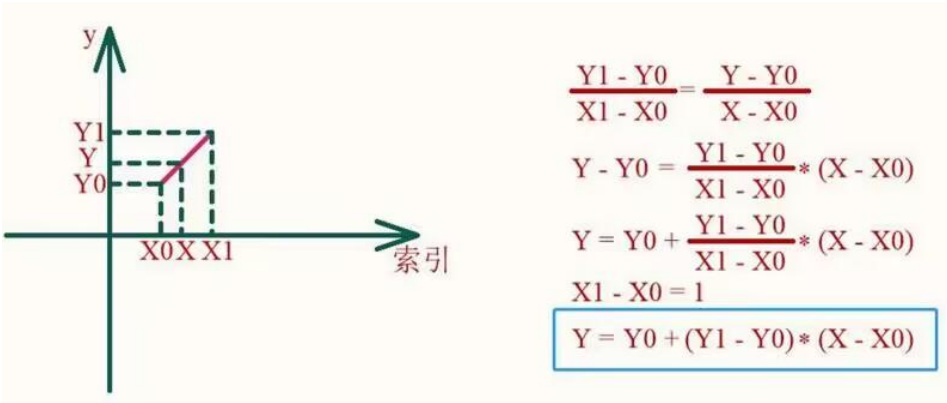
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
//Sin
Temp = DSRPAG;
DSRPAG = __builtin_psvpage(SinTable);
p1 = (unsigned int *)__builtin_psvoffset(SinTable);
p2 = p1 + tCore.Index.Words.High;
Y0 = *p2;
TempIndex = tCore.Index.Words.High + 1;
if(TempIndex > 127)
    TempIndex = TempIndex - 128;
p2 = p1 + TempIndex;
Y1 = *p2;
tCore.qSin = Y0 + (__builtin_mulus(tCore.Index.Words.Low,(Y1 - Y0)) >> 16);
```

然后，索引号加 1，就把后面一个正弦值给查表得到，也就对应程序中的 Y1。

```
//Sin
Temp = DSRPAG;
DSRPAG = __builtin_psvpage(SinTable);
p1 = (unsigned int *)__builtin_psvoffset(SinTable);
p2 = p1 + tCore.Index.Words.High;
Y0 = *p2;
TempIndex = tCore.Index.Words.High + 1;
if(TempIndex > 127)
    TempIndex = TempIndex - 128;
p2 = p1 + TempIndex;
Y1 = *p2;
tCore.qSin = Y0 + (__builtin_mulus(tCore.Index.Words.Low,(Y1 - Y0)) >> 16);
```

最后呢，用我们之前讲过的公式，计算一下插值，就得到了对应的插值计算的正弦值。



这是我们之前讲过的公式啊。X-X0 呢，就是索引的小数部分，也就是索引号的低 16 位。根据我们之前讲过的运算法则，两个混合符号的数相乘，乘完之后变为 Q15，就要右移 16 位然后，同理呢，我们可以把余弦值也求取出来。

```
//Cos
TempIndex = tCore.Index.Words.High + 32;
if(TempIndex > 127)
    TempIndex = TempIndex - 128;
p2 = p1 + TempIndex;
Y0 = *p2;
TempIndex += 1;
if(TempIndex > 127)
    TempIndex = TempIndex - 128;
p2 = p1 + TempIndex;
Y1 = *p2;
tCore.qCos = Y0 + (__builtin_mulus(tCore.Index.Words.Low,(Y1 - Y0)) >> 16);
DSRPAG = Temp;
```

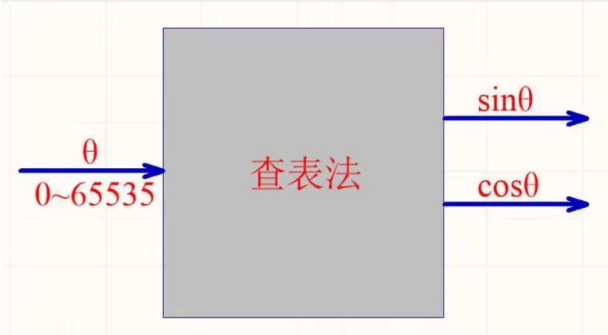
思路呢，还是先把索引号加 32.　然后查出余弦值对应的 Y0　Y1。然后，再根据插值公式，计算出此时的余弦值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else
{
    //Sin
    Temp = DSRPAG;
    DSRPAG = __builtin_psvpage(SinTable);
    p1 = (unsigned int *)__builtin_psvoffset(SinTable);
    p2 = p1 + tCore.Index.Words.High;
    Y0 = *p2;
    TempIndex = tCore.Index.Words.High + 1;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    Y1 = *p2;
    tCore.qSin = Y0 + (__builtin_mulus(tCore.Index.Words.Low,(Y1 - Y0)) >> 16);

    //Cos
    TempIndex = tCore.Index.Words.High + 32;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    Y0 = *p2;
    TempIndex += 1;
    if(TempIndex > 127)
        TempIndex = TempIndex - 128;
    p2 = p1 + TempIndex;
    Y1 = *p2;
    tCore.qCos = Y0 + (__builtin_mulus(tCore.Index.Words.Low,(Y1 - Y0)) >> 16);
    DSRPAG = Temp;
}
```

然后，同样恢复现场，这个函数就结束了。



以上呢，就是我们分析的查表法+线性插值的代码。关于我们今天分析的代码，大家有什么问题？

问：这个和汇编的对比时间上能差多少？
答：没有对比过呢。

问：

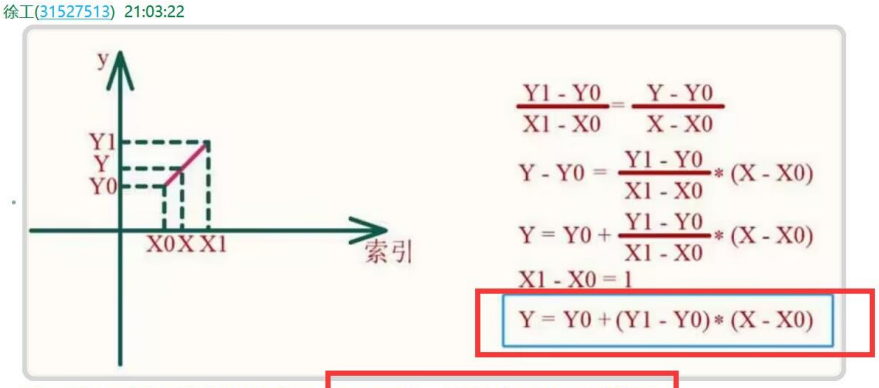
```
//Sin
Temp = DSRPAG;
DSRPAG = __builtin_psvpage(SinTable);
p1 = (unsigned int *)__builtin_psvoffset(SinTable);
p2 = p1 + tCore.Index.Words.High;
Y0 = *p2;
TempIndex = tCore.Index.Words.High + 1;
if(TempIndex > 127)
    TempIndex = TempIndex - 128;
p2 = p1 + TempIndex;
Y1 = *p2;
tCore.qSin = Y0 + (__builtin_mulus(tCore.Index.Words.Low,(Y1 - Y0)) >> 16);
```

这个函数功能是干嘛的，能讲一下吗？

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

答：

徐工(31527513) 21:03:22

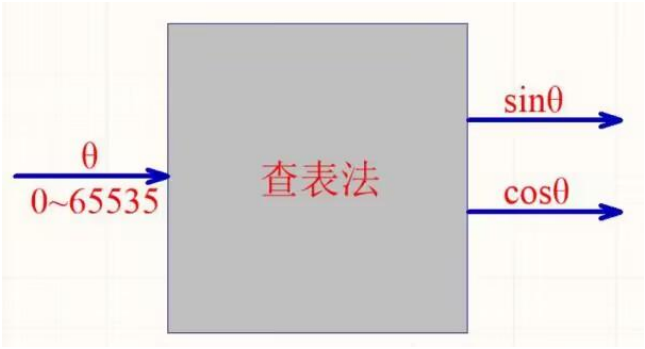


这是我们之前讲过的公式啊，X-X0呢，就是索引的小数部分

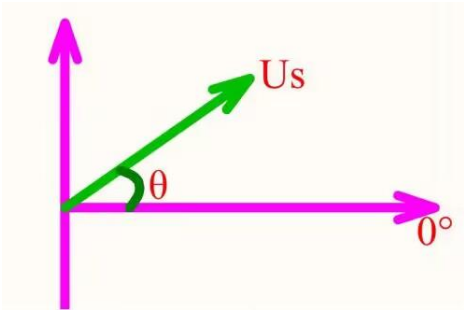
Q16 * Q15 = Q31，Q31 右移 16 位得到 Q15。

问：这个内建函数，在 dsp 其他型号也是这么用的？还是只有这款芯片这么用？
答：只有 Microchip 部分系列的单片机可以使用。

好了，我们继续来分析啊



查表法清楚了，也就是说我们单片机中求取 sin θ，cos θ 的方式就清楚了。但是，我们继续往前推的话，现在的问题我们不清楚这个 θ 是怎么来的。所以呢，我们下一步的目标，就是要想办法把这个 θ 给求取出来。要求取这个 θ，我们就要知道这个 θ 的含义。之前呢，我们讲过，这个 θ，它是电压合成矢量跟 0° 方向的夹角。



大家看，假设 Us 是电压合成矢量，那么这个电压合成矢量跟 0° 方向的夹角就是图中的 θ。我们现在的目标，就是把这个 θ 给求取出来。求取出来之后，才能得到对应的 sin θ，跟 cos θ。那关键是，要求这个 θ 的话，仅仅从上图，似乎也不能把 θ 求解出来。我们要想办法，让这个 θ 跟其它的量建立一个联系。例如，我们电机安装有霍尔传感器，安装霍尔传感器的话，可以把转子位置解算出来。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

那么，如果我们能够找到转子位置跟这个 θ 的关系，知道了转子的位置，也自然就知道了这个 θ 。它们之间，有什么关系呢？大家课后下去思考思考，我们下节课再来分析这部分我们需要先从有传感来理解一下，后面逐渐过渡到无感。那我们今天就先讲到这里啊，大家有没有问题？

问：这个 Us 和 Ualpha、Ubeta、Ud、Uq 的关系能说说不？
答：Ualpha、Ubeta、Ud、Uq 分别是 Us 在 alpha，beta，d,q 轴上的投影。

大家早点休息，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。